

Rol del programador en la era de la IA y Code Health Guardian

En la actualidad, con el avance de herramientas de Inteligencia Artificial como Copilot o modelos generativos, el papel del programador está cambiando. Según Artie Shevchenko, el desarrollador no debe limitarse a colaborar y delegar tareas a la IA, sino que debe destacar en algo en lo que la IA todavía tiene dificultades: la gestión de la complejidad del código.

Para ello introduce el concepto de Code Health Guardian, que define al programador como el responsable de mantener el código simple, mantenible y comprensible a largo plazo. La IA puede generar soluciones rápidas y aparentemente correctas, pero tiende a producir soluciones simplistas en lugar de verdaderamente simples, lo que puede aumentar la complejidad del sistema con el tiempo.

Uno de los puntos clave es el equilibrio entre productividad a corto y largo plazo. Si un equipo se enfoca únicamente en el corto plazo, puede generar soluciones rápidas pero llenas de deuda técnica, aumentando la complejidad futura. En cambio, si se enfoca excesivamente en el largo plazo, puede caer en la sobreingeniería y retrasar la entrega de valor al usuario. El punto óptimo se encuentra en un balance entre ambas perspectivas.

Otro aspecto importante es la mentalidad de ownership. El desarrollador debe actuar como si el producto y los recursos fueran propios, tomando decisiones pensando en el impacto real en el usuario y en la sostenibilidad del proyecto. No basta con escribir código que funcione; es necesario escribir código que pueda mantenerse y evolucionar con el tiempo.

Complejidad del código y sus problemas

Según John Osterhout existen tres problemas distintos de complejidad, los cuales son Change Amplification, Cognitive Load y Unknown Unknowns.

Change Amplification habla de un pequeño cambio que se realice en un elemento puede dar a lugar a muchos cambios grandes en diferentes elementos. Es el más leve, porque suele afectar solo a la velocidad del desarrollo más que a los errores que puedan surgir.

Cognitive Load es el esfuerzo mental que se tiene que hacer para entender el código. Este problema es más grave, ya que, a mayor carga cognitiva, más tiempo se va a tardar en entender el proyecto, y esto puede conllevar a una tasa de errores mayor.

Por último, se tiene el *Unknow Unknowns*, que consiste en el desconocimiento de información importante que se necesita para realizar correctamente el trabajo. Es el más grave porque es el que más desapercibido puede pasar y el que genera bugs importantes.

Estos problemas aparecen frecuentemente por la obscuridad, código duplicado, interfaces complejas y falta de consistencia, donde el peor es la obscuridad.

Para minimizarlos, tenemos las opciones de el diseño por contrato, las aserciones y el principio DRY con equilibrio.

Herramientas para gestionar la complejidad: revisiones de código, documentos de diseño y trade-offs

Tras analizar los problemas derivados de la complejidad del código, surge la necesidad de aplicar prácticas que permitan controlarla de forma activa. El podcast plantea que mantener la salud del código no depende únicamente del trabajo individual del desarrollador, sino de mecanismos colectivos que ayuden a tomar mejores decisiones técnicas. Entre ellos destacan los documentos de diseño, las revisiones de código y la comprensión consciente de los *trade-offs*.

Los documentos de diseño permiten abordar la complejidad antes incluso de comenzar la implementación. Su función principal es hacer explícitas las decisiones técnicas, las alternativas consideradas y sus limitaciones, favoreciendo discusiones estructuradas dentro del equipo. En este contexto, los *trade-offs* adquieren especial relevancia, ya que cada decisión implica equilibrar objetivos como velocidad de desarrollo, simplicidad, flexibilidad o rendimiento. Registrar estos compromisos resulta clave, pues documenta no solo qué decisión se tomó, sino también el razonamiento detrás de ella, reduciendo la ambigüedad futura y la carga cognitiva al volver sobre el sistema.

Las revisiones de código complementan este proceso durante el desarrollo. Más allá de detectar errores, permiten comprobar si la implementación respeta las decisiones de diseño y evita introducir complejidad innecesaria. Además, fomentan la compartición de conocimiento y reducen la dependencia de personas concretas, mitigando la aparición de *unknown unknowns*. Cuando se realizan en un entorno constructivo, también contribuyen a la seguridad psicológica del equipo, convirtiéndose en un espacio de aprendizaje donde las mejoras se enfocan en el código y no en la persona.

En conjunto, estas prácticas refuerzan una visión del desarrollo centrada en la salud del código, donde equilibrar productividad inmediata y sostenibilidad futura permite mantener sistemas más comprensibles y fáciles de evolucionar.